

Design and Verification of a Single-Cycle and a Pipelined RV32I RISC-V Processor

Final Project Report

Prepared by:

Nour Ahmed Mahmoud Foad
Yassin Wael Shawky

Abstract

This report documents the design, implementation, and verification of a 32-bit RV32I RISC-V processor, developed in two stages. The first stage is a single-cycle microarchitecture built on a Harvard-style datapath, in which every instruction completes fetch, decode, execute, memory access, and write-back within one clock period. The second stage restructures the same datapath into a classic five-stage pipeline (IF, ID, EX, MEM, WB), adding pipeline registers, a forwarding unit, a hazard detection unit, and branch-flush logic to preserve correct program behaviour while increasing instruction throughput. Both designs were implemented in synthesizable Verilog/SystemVerilog, driven by the same instruction set subset (R-type ALU operations, I-type ALU operations, loads, stores, and the `beq/bne/blt` branches), and verified against a Fibonacci-series test program. This report explains the operating principle of each microarchitecture, presents block diagrams of both datapaths, documents every functional unit and the control logic that drives it, walks through the hazard-handling mechanisms added for the pipelined version, and closes with a structured comparison of the two designs in terms of cycle time, CPI, throughput, and hardware cost.

Contents

Abstract	1
1 Introduction	6
1.1 Project Overview	6
1.2 Harvard Architecture	6
1.3 Report Organization	6
2 Single-Cycle RV32I Processor	7
2.1 Basic Idea	7
2.2 Datapath Block Diagram	7
2.3 Functional Units	8
2.3.1 Program Counter (PC)	8
2.3.2 Instruction Memory	8
2.3.3 Register File	8
2.3.4 Sign / Immediate Extender	9
2.3.5 ALU	9
2.3.6 Data Memory	9
2.3.7 2:1 Multiplexer	9
2.4 Control Unit	9
2.4.1 Branch Decision	10
3 Pipelined RV32I Processor	12
3.1 Motivation	12
3.2 Five-Stage Pipeline Overview	12
3.3 Pipeline Registers	13
3.4 Hazards and Their Resolution	13
3.4.1 Structural Hazards	13
3.4.2 Data Hazards – Forwarding	14
3.4.3 Load-Use Hazard – Stalling	14
3.4.4 Control Hazards – Branch Resolution	14
3.5 Signal Naming: Handout vs. Implementation	14
4 Verification and Simulation Results	16
4.1 Test Program	16
4.2 Single-Cycle Simulation	17
4.3 Pipelined Simulation	17
4.4 Cross-Check	19
5 Single-Cycle vs. Pipelined: Comparison	20
5.1 Qualitative Comparison	20
5.2 Quantitative Comparison	20
5.3 Discussion	21

6	Design Notes, Assumptions, and Known Limitations	22
7	Conclusion	23
A	Module Interfaces	24
B	Selected RTL Listings	25
B.1	Hazard Detection Unit	25
B.2	Forwarding Unit	25
B.3	Branch Unit	26

List of Figures

2.1	Single-cycle RV32I datapath (Harvard architecture).	7
2.2	Program-counter block: the next value of the PC is chosen between $PC + 4$ (sequential execution) and the branch target computed in EX.	8
2.3	Register-file ports. Reads are asynchronous (combinational); the write on <code>WD3</code> is synchronous and only happens when <code>WE3</code> is asserted and <code>A3</code> $\neq 0$	8
2.4	Control-unit block diagram: the main decoder produces the datapath-wide control word directly from the opcode, while the ALU decoder derives the 3-bit <code>ALUControl</code> field from <code>ALUOp</code> together with <code>funct3/funct7</code> so that, e.g., <code>add</code> and <code>sub</code> (same opcode, same <code>funct3</code>) can still be told apart.	11
3.1	Five-stage pipelined RV32I datapath with pipeline registers, the forwarding unit, and the hazard detection unit.	13
4.1	Single-cycle simulation waveform for the Fibonacci program.	17
4.2	Pipelined simulation waveform (IF/ID/EX signals).	18
4.3	Pipelined simulation waveform (EX/MEM/WB signals, forwarding, and hazard signals).	18

List of Tables

2.1	Immediate-extend encoding (ImmSrc).	9
2.2	ALU control encoding.	9
2.3	Main decoder truth table.	10
2.4	ALU decoder truth table.	10
3.1	Fields carried by each pipeline register.	13
3.2	Forwarding encoding (applies independently to ForwardA and ForwardB).	14
3.3	Signal name mapping between the handout's single-cycle figure and the pipelined RTL.	15
4.1	Fibonacci test program (RISC-V assembly).	16
5.1	Structural comparison of the two microarchitectures.	20
A.1	Summary of RTL modules and their role in the pipeline.	24

Chapter 1

Introduction

1.1 Project Overview

The goal of this project is to implement a 32-bit RV32I processor twice: once as a **single-cycle** machine, and once as a **5-stage pipelined** machine built by re-using the single-cycle functional units (ALU, register file, memories, sign extender, control unit) and inserting pipeline registers and hazard-resolution hardware between the stages. Both machines implement the same subset of the RV32I instruction set:

- R-type ALU instructions: `add`, `sub`, `xor`, `or`, `and`, shift-left-logical, shift-right-logical.
- I-type ALU instructions (immediate form of the above, e.g. `addi`, `slli`).
- `lw` (load word) and `sw` (store word).
- Conditional branches `beq`, `bne`, `blt`.

1.2 Harvard Architecture

Both designs use a **Harvard** organization: the instruction memory and the data memory are two physically separate 32-entry-deep, 32-bit-wide, word-addressed memories. Because the two memories are independent, an instruction fetch and a data-memory access can happen in the same clock edge without contention – this is what makes it possible for the pipelined version to overlap the IF stage of one instruction with the MEM stage of another without a structural hazard on memory.

1.3 Report Organization

Chapter 2 describes the single-cycle datapath and its control logic. Chapter 3 describes how that datapath was converted into a 5-stage pipeline, including the pipeline registers, the forwarding unit, the hazard detection unit, and branch resolution. Chapter 4 presents the verification methodology and waveform results for both machines on the same test program. Chapter 5 compares the two microarchitectures quantitatively. Chapter 6 records the design assumptions, edge cases, and known limitations. Chapter 7 concludes the report.

Chapter 2

Single-Cycle RV32I Processor

2.1 Basic Idea

In the single-cycle microarchitecture, an entire instruction – **fetch, decode, execute, memory access, and write-back** – happens within a single clock period. There is exactly one instance of every functional unit (one ALU, one register file, one instruction memory, one data memory), and there are no pipeline registers: the outputs of one functional unit feed combinationally into the inputs of the next, all within the same cycle. The clock period must therefore be at least as long as the *longest* possible instruction path (in practice the `lw` path: instruction fetch → register read → ALU address computation → data memory read → register write-back).

2.2 Datapath Block Diagram

Figure 2.1 shows the complete single-cycle datapath: the program counter, instruction memory, register file, sign extender, ALU, data memory, and the control unit that drives every mux and enable signal in the datapath.

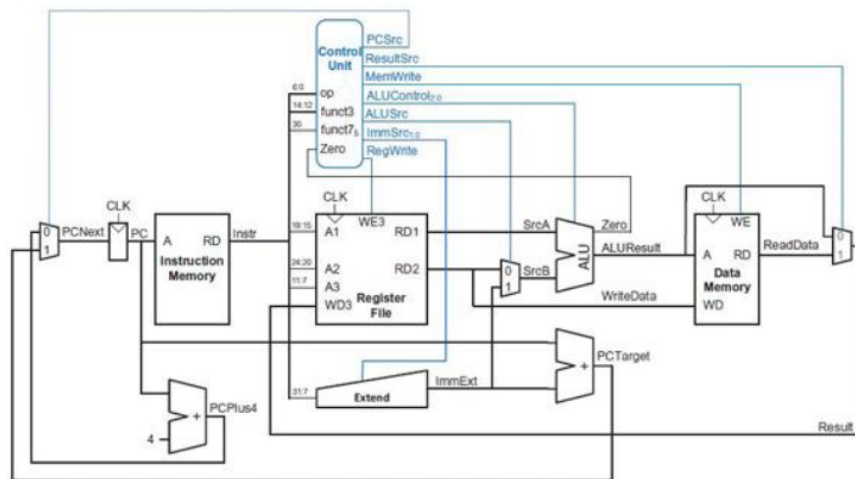


Figure 2.1: Single-cycle RV32I datapath (Harvard architecture).

2.3 Functional Units

2.3.1 Program Counter (PC)

The PC is a 32-bit register with an asynchronous, active-low reset and a synchronous load-enable (`load`). `load` is held high at all times except when a `halt` condition is detected, in which case the PC is frozen so the processor stops fetching new instructions. The next-PC value is chosen by a 2:1 mux:

$$\text{PCNext} = \begin{cases} \text{PC} + 4 & \text{PCSrc} = 0 \\ \text{PC} + \text{ImmExt} & \text{PCSrc} = 1 \text{ (branch taken)} \end{cases}$$

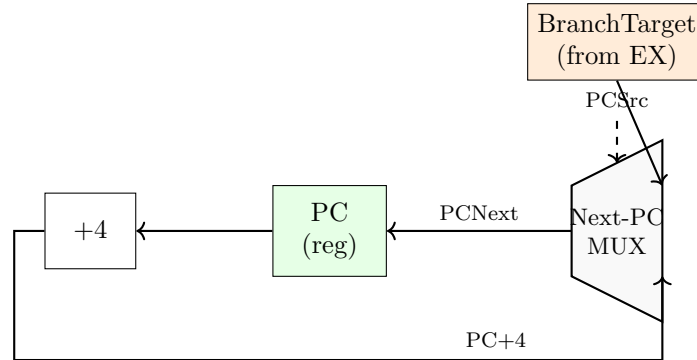


Figure 2.2: Program-counter block: the next value of the PC is chosen between $\text{PC}+4$ (sequential execution) and the branch target computed in EX.

2.3.2 Instruction Memory

A 64×32 -bit, word-addressed, asynchronously-read ROM. Only bits [31:2] of the address are used since instructions are always word-aligned. It is pre-loaded with the program image (e.g. via `$readmemh`).

2.3.3 Register File

A 32×32 -bit register bank with two asynchronous read ports (`RD1`, `RD2`) and one synchronous write port (`WD3`, gated by `WE3`), with an active-low asynchronous reset that clears all 32 registers. Register `x0` is hard-wired to read as zero.

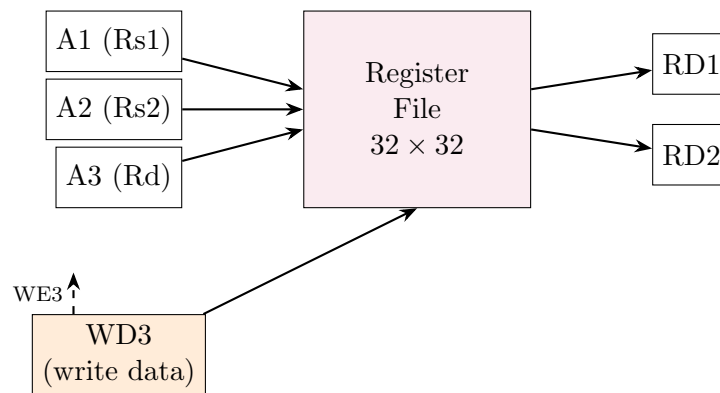


Figure 2.3: Register-file ports. Reads are asynchronous (combinational); the write on `WD3` is synchronous and only happens when `WE3` is asserted and `A3` $\neq 0$.

2.3.4 Sign / Immediate Extender

Produces a 32-bit sign-extended immediate from the instruction, selecting among the I-type, S-type, and B-type immediate encodings using the 2-bit `ImmSrc` control signal (Table 2.1).

Table 2.1: Immediate-extend encoding (`ImmSrc`).

<code>ImmSrc</code>	<code>ImmExt</code>	Type
00	$\{20\{\text{Instr}[31]\}, \text{Instr}[31:20]\}$	I – 12-bit signed immediate
01	$\{20\{\text{Instr}[31]\}, \text{Instr}[31:25], \text{Instr}[11:7]\}$	S – 12-bit signed immediate
10	$\{20\{\text{Instr}[31]\}, \text{Instr}[7], \text{Instr}[30:25], \text{Instr}[11:8], 1'b0\}$	B – 13-bit signed immediate

2.3.5 ALU

A purely combinational unit. `ALUcontrol` selects the operation (Table 2.2); it also produces a **zero** flag (`ALU_Out == 0`) and a **sign** flag (`ALU_Out[31]`), both of which feed the branch-decision logic in the control unit.

Table 2.2: ALU control encoding.

<code>ALUControl</code>	Function
000	$A + B$ (add)
001	$A \ll B$ (shift left logical)
010	$A - B$ (subtract)
100	$A \oplus B$ (xor)
101	$A \gg B$ (shift right logical)
110	$A B$ (or)
111	$A \& B$ (and)
Otherwise	0 (default)

2.3.6 Data Memory

A 64×32 -bit memory with a single read/write port. Reads are asynchronous; writes are synchronous, occurring on the rising clock edge when `WE` is asserted.

2.3.7 2:1 Multiplexer

A single reusable 32-bit-wide 2:1 mux module ($Z = S0 ? B : A$) is instantiated wherever the datapath needs to select between two 32-bit sources (ALU operand B, write-back result, next-PC source).

2.4 Control Unit

The control unit is factored into two combinational blocks, following the classic Patterson & Hennessy structure:

- **Main decoder** – maps the 7-bit opcode to `RegWrite`, `ImmSrc`, `ALUSrc`, `MemWrite`, `ResultSrc`, `Branch`, and `ALUOp`.
- **ALU decoder** – maps `ALUOp`, `funct3`, and `funct7` to the 3-bit `ALUControl` signal.

Table 2.3: Main decoder truth table.

Opcode	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp
lw (0000011)	1	00	1	0	1	0	00
sw (0100011)	0	01	1	1	X	0	00
R-type (0110011)	1	XX	0	0	0	0	10
I-type (0010011)	1	00	1	0	0	0	10
branch (1100011)	0	10	0	0	X	1	01
default	0	00	0	0	0	0	00

Table 2.4: ALU decoder truth table.

ALUOp	funct3	{op[5],funct7}	ALUControl	Instruction
00	XXX	XX	000 (add)	lw, sw
	000	XX	010 (sub)	beq
01	001	XX	010 (sub)	bne
	100	XX	010 (sub)	blt
	000	00,01,10	000 (add)	add
	000	11	010 (sub)	sub
10	001	XX	001 (shl)	slli
	100	XX	100 (xor)	xor
	101	XX	101 (shr)	srli
	110	XX	110 (or)	or
	111	XX	111 (and)	and
default	XXX	XX	000	—

2.4.1 Branch Decision

Branch instructions all share opcode 1100011; **funct3** distinguishes which comparison to perform, and the ALU is used in subtract mode to produce the **zero** and **sign** flags:

- beq: PCSrc = Branch & zero
- bne: PCSrc = Branch & ~zero
- blt: PCSrc = Branch & sign

A small mux/case block selects among these three conditions based on **funct3**, and its output directly drives the PC-select mux.

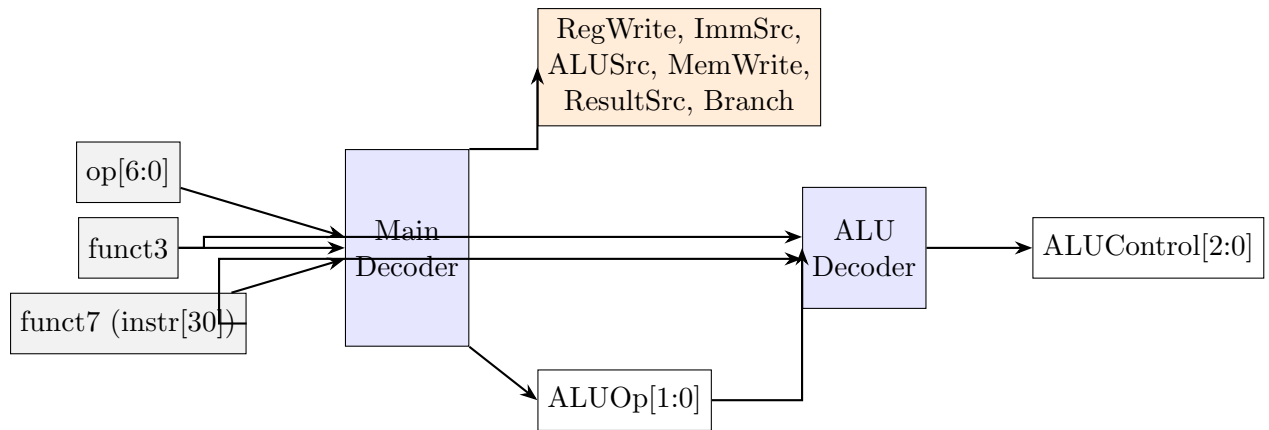


Figure 2.4: Control-unit block diagram: the main decoder produces the datapath-wide control word directly from the opcode, while the ALU decoder derives the 3-bit **ALUControl** field from **ALUOp** together with **funct3/funct7** so that, e.g., **add** and **sub** (same opcode, same **funct3**) can still be told apart.

Chapter 3

Pipelined RV32I Processor

3.1 Motivation

In the single-cycle design, the clock period is bounded by the slowest instruction's end-to-end combinational delay, even though most instructions do not use every functional unit. Pipelining splits the datapath into stages separated by registers, so that the clock period only needs to cover the *slowest single stage* rather than the *sum of all stages*. Up to five instructions can then be in flight simultaneously, which increases instruction throughput even though the latency of any single instruction (in clock cycles) is unchanged or slightly higher.

3.2 Five-Stage Pipeline Overview

The datapath is split into the following stages, matching the RTL module names used in the implementation:

1. **IF** – Instruction Fetch (`PC_reg`, `instr_mem`)
2. **ID** – Instruction Decode / Register Read (`register_file`, `sign_extend`, `control_unit`)
3. **EX** – Execute (`ALU`, `branch_unit`, `forward_unit`)
4. **MEM** – Memory Access (`data_mem`)
5. **WB** – Write Back (result mux back into `register_file`)

Four pipeline registers hold the state and control signals flowing between consecutive stages: `if_id_reg`, `id_ex_reg`, `ex_mem_reg`, and `mem_wb_reg`.

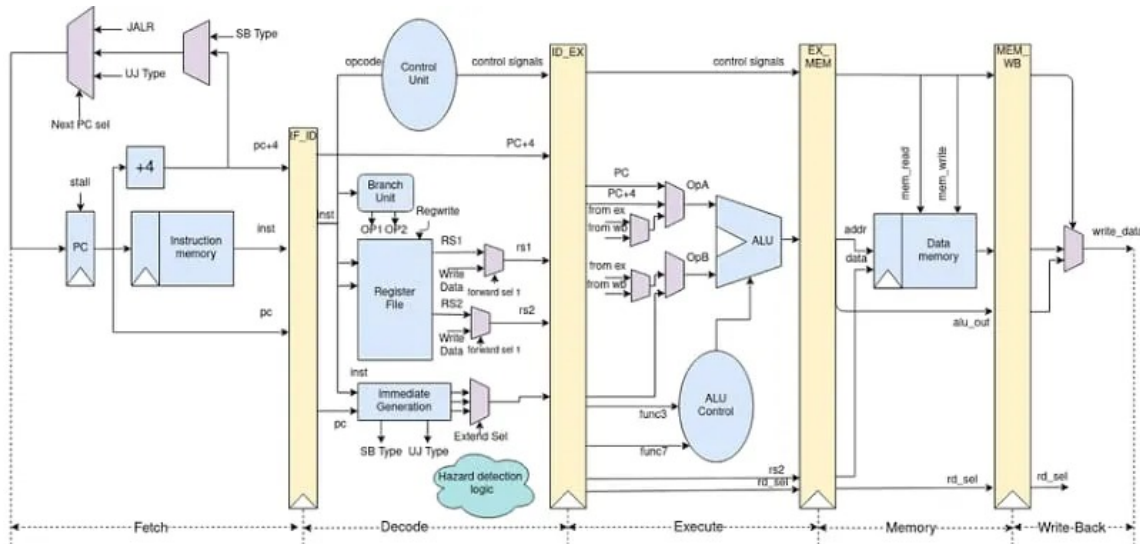


Figure 3.1: Five-stage pipelined RV32I datapath with pipeline registers, the forwarding unit, and the hazard detection unit.

3.3 Pipeline Registers

Each pipeline register carries forward both datapath values and the already-decoded control signals for the instruction currently occupying that stage, so that every downstream stage can act on an instruction without re-decoding it. All four registers share the same reset/stall/flush discipline:

- **Reset** (`areset` low): synchronously clears every field to zero.
- **Flush**: clears only the *control* fields relevant to that stage (e.g. `RegWrite`, `MemWrite`, `Branch`, `Rd`) to zero, turning the in-flight instruction into an inert bubble while leaving data fields undefined (their control is disabled, so their data cannot affect architectural state).
- **Stall**: holds the register's current outputs unchanged for one cycle (used only by `if_id_reg` and `id_ex_reg`).

Table 3.1: Fields carried by each pipeline register.

Register	Data fields	Control fields
IF/ID	Instr, PC	—
ID/EX	PC, RD1, RD2, ImmExt, Rd, Rs1, Rs2, funct3	ALUSrc, ALUControl, MemWrite, RegWrite, Branch, Rd
EX/MEM	ALUResult, WriteData, Rd	MemWrite, RegWrite, ResultSrc
MEM/WB	ALUResult, ReadData, Rd	RegWrite, ResultSrc

3.4 Hazards and Their Resolution

3.4.1 Structural Hazards

Because the design is Harvard (separate instruction and data memories) and the register file supports simultaneous read and write in the same cycle, no structural hazard exists between the IF stage of a later instruction and the MEM/WB stage of an earlier one – both memories and the register file can be accessed every cycle without contention.

3.4.2 Data Hazards – Forwarding

When an instruction in EX needs an operand that has not yet been written back to the register file (because the producing instruction is still in MEM or WB), the `forward_unit` detects the dependency and routes the correct value into the ALU inputs instead of the stale register-file output. The forwarding priority favours the closer (EX/MEM) producer over the older (MEM/WB) producer:

Table 3.2: Forwarding encoding (applies independently to ForwardA and ForwardB).

Forward code	Meaning
00	No forwarding – use RD1_EX / RD2_EX as decoded
10	Forward from EX/MEM (<code>ALUResult_MEM</code>)
01	Forward from MEM/WB (<code>Result</code> , the write-back value)

Forwarding condition for operand A (rs1), analogous for operand B (rs2):

$$\text{ForwardA} = \begin{cases} 10 & \text{RegWrite_MEM} \wedge (\text{Rd_MEM} \neq 0) \wedge (\text{Rd_MEM} = \text{Rs1_EX}) \\ 01 & \text{RegWrite_WB} \wedge (\text{Rd_WB} \neq 0) \wedge (\text{Rd_WB} = \text{Rs1_EX}) \\ 00 & \text{otherwise} \end{cases}$$

3.4.3 Load-Use Hazard – Stalling

Forwarding alone cannot resolve a load-use hazard, because the loaded value is not available until the end of the MEM stage – one cycle too late for an immediately-following dependent instruction in EX. The `hazard_detection_unit` detects this specific case:

$$\text{lwStall} = \text{ResultSrc_EX} \wedge (\text{Rd_EX} \neq 0) \wedge [(\text{Rd_EX} = \text{Rs1_ID}) \vee (\text{Rd_EX} = \text{Rs2_ID})]$$

When asserted, `lwStall` freezes the PC and the IF/ID register for one cycle and injects a bubble into ID/EX (by flushing its control fields), which delays the dependent instruction by exactly one cycle – long enough for the load’s result to become forward-able from EX/MEM on the following cycle.

3.4.4 Control Hazards – Branch Resolution

Branches are resolved in the **EX** stage (`branch_unit`), using the ALU’s `zero/sign` flags computed for that instruction. By the time a branch’s outcome (`PCSrc_EX`) is known, two younger instructions have already been fetched into IF/ID and ID/EX. If the branch is taken, both of these are squashed:

- `if_id_reg` is flushed (cleared) so the wrongly-fetched instruction in IF/ID does not proceed to decode.
- `id_ex_reg` is flushed as well (via `PCSrc_EX || lwStall`), so the instruction that was in ID during the branch’s EX cycle also becomes a bubble.

This gives a fixed **2-cycle branch misprediction penalty** – there is no branch predictor; every branch is effectively ”predicted not-taken” and corrected in EX if it resolves taken.

3.5 Signal Naming: Handout vs. Implementation

The project handout’s datapath figure and this implementation use slightly different signal names for the same quantities (e.g. the handout’s `SrcA/SrcB` correspond to the ALU inputs,

which in the pipelined RTL become **ForwardedA**/**SrcB** once forwarding muxes are inserted). Table 3.3 records the mapping used throughout this report and the RTL.

Table 3.3: Signal name mapping between the handout’s single-cycle figure and the pipelined RTL.

Handout (single-cycle)	Pipelined RTL
PCNext / PCTarget	BranchTarget_EX (computed as PC_EX + ImmExt_EX)
SrcA	ForwardedA (RD1_EX, or forwarded value)
SrcB (before ALUSrc mux)	ForwardedB (RD2_EX, or forwarded value)
SrcB (after ALUSrc mux)	SrcB (mux of ForwardedB and ImmExt_EX)
ALUResult	ALUResult → ALUResult_MEM → ALUResult_WB
ReadData	ReadData → ReadData_WB
Result (write-back mux)	Result (mux of ALUResult_WB and ReadData_WB)
Zero	Zero (from ALU, registered implicitly by ID/EX before EX use)

Chapter 4

Verification and Simulation Results

4.1 Test Program

Both processors were verified using the same assembly program: an iterative computation of the first ten terms of the Fibonacci sequence, alternating between two accumulator registers (**x1**, **x2**) selected by a toggling flag (**x3**), storing each new term to data memory. The program exercises every instruction class supported by the design: **addi** (I-type ALU), **add/sub** (R-type ALU), **slli** (R-type shift), **sw** (store), **beq** (equality branch), and **blt** (less-than branch).

Table 4.1: Fibonacci test program (RISC-V assembly).

Address	Instruction	Comment
0	xor x0,x0,x0	reference register, always 0
4	addi x1,x0,0	data register (Fibonacci value)
8	addi x2,x0,1	data register (Fibonacci value)
12	addi x3,x0,1	selector
16	addi x4,x0,1	constant 1
20	addi x5,x0,0	loop counter
24	addi x6,x0,10	total loop count
28	addi x7,x0,0	data-memory base address
32	beq x3,x4,eq	branch if selector == 1
36	add x2,x2,x1	
40	sub x3,x3,x4	
44	slli x7,x5,2	
48	sw x2,0(x7)	
52	beq x4,x4,endloop	unconditional jump
56	add x1,x1,x2	(eq:)
60	add x3,x3,x4	
64	slli x7,x5,2	
68	sw x1,0(x7)	
72	addi x5,x5,1	(endloop:)
76	blt x5,x6,loop	
80	halt	optional: freeze PC

The expected sequence of values stored to data memory is the Fibonacci sequence 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, which is the golden reference used to validate both simulations.

4.2 Single-Cycle Simulation

Figure 4.1 shows the single-cycle simulation waveform. Each instruction completes in exactly one clock cycle, so the data-memory contents (`mem[0]` through `mem[9]`) update to consecutive Fibonacci terms one cycle apart, matching the expected output supplied with the project handout.

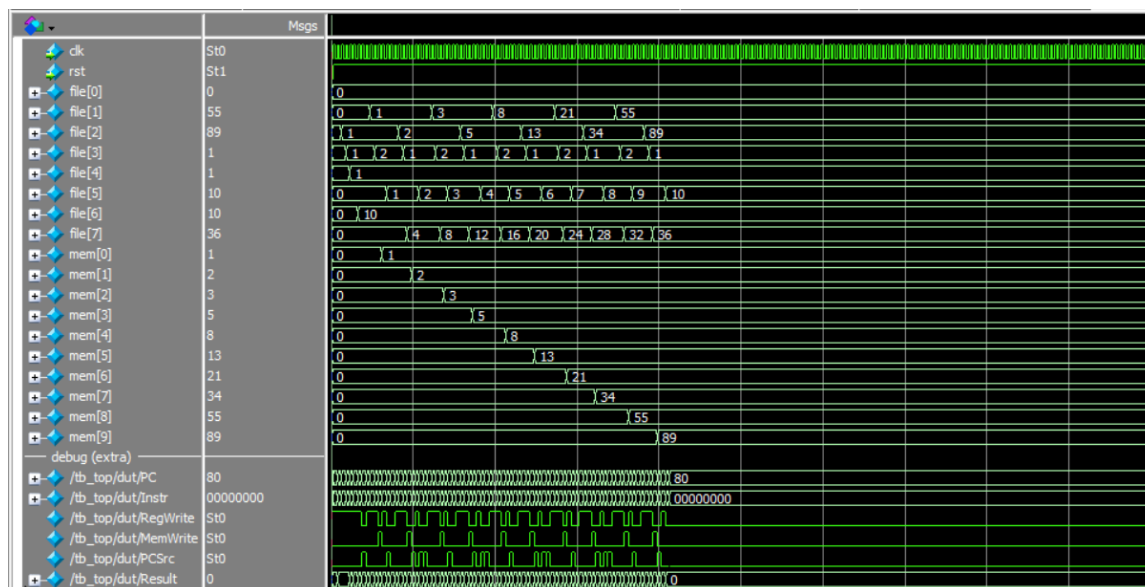


Figure 4.1: Single-cycle simulation waveform for the Fibonacci program.

4.3 Pipelined Simulation

Figures 4.2 and 4.3 show the pipelined simulation. Unlike the single-cycle waveform, each signal now appears staggered by stage: e.g. PC advances in IF while the control signals for the *same* instruction only appear one cycle later on the `_EX` signals, two cycles later on the `_MEM` signals, and three cycles later on the `_WB` signals. The waveform also shows:

- `ForwardA/ForwardB` switching to `2'b10` or `2'b01` exactly when a dependent instruction reaches EX one or two cycles after its producer, confirming the forwarding paths are functioning.
- `lwStall` pulsing high for one cycle after a `lw` is immediately followed by a dependent instruction, confirming the load-use hazard is caught and stalled.
- `PCSrc_EX` pulsing high on a taken branch, coinciding with the following cycle's `Instr_reg` and `PC_EX` being flushed to zero, confirming the 2-bubble branch flush.

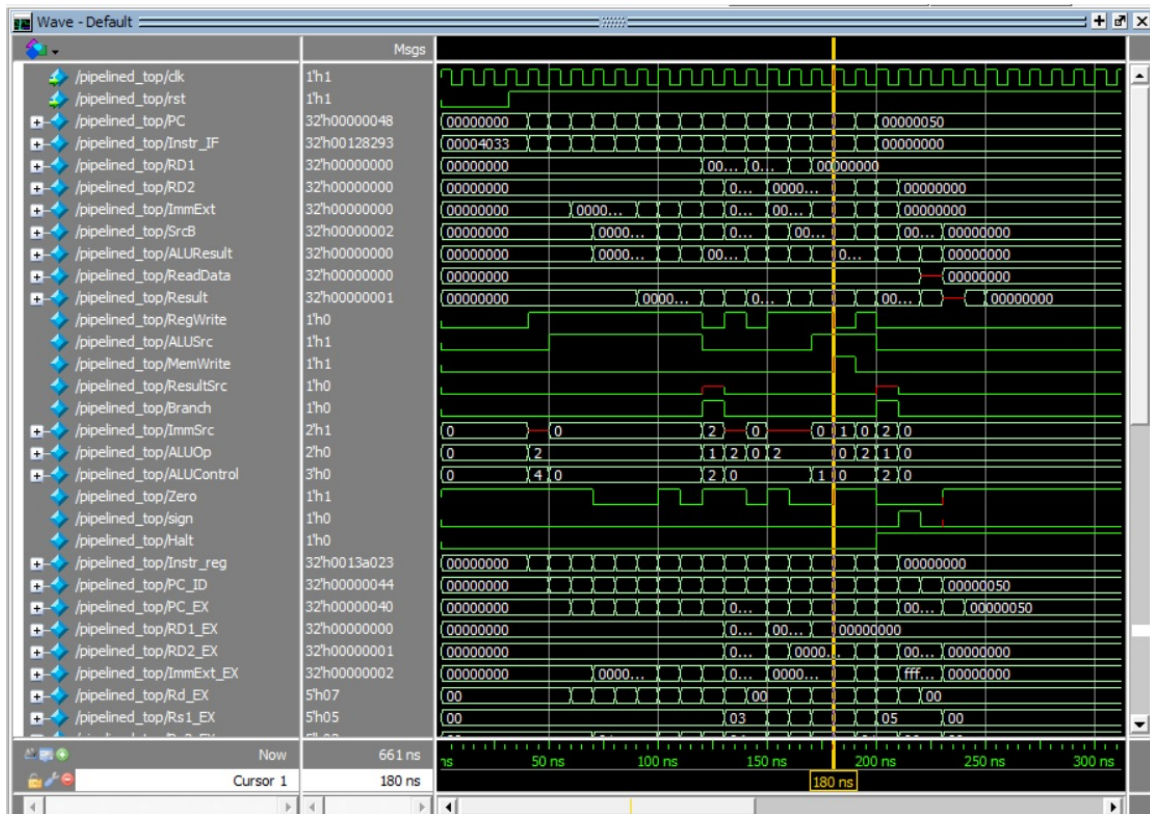


Figure 4.2: Pipelined simulation waveform (IF/ID/EX signals).

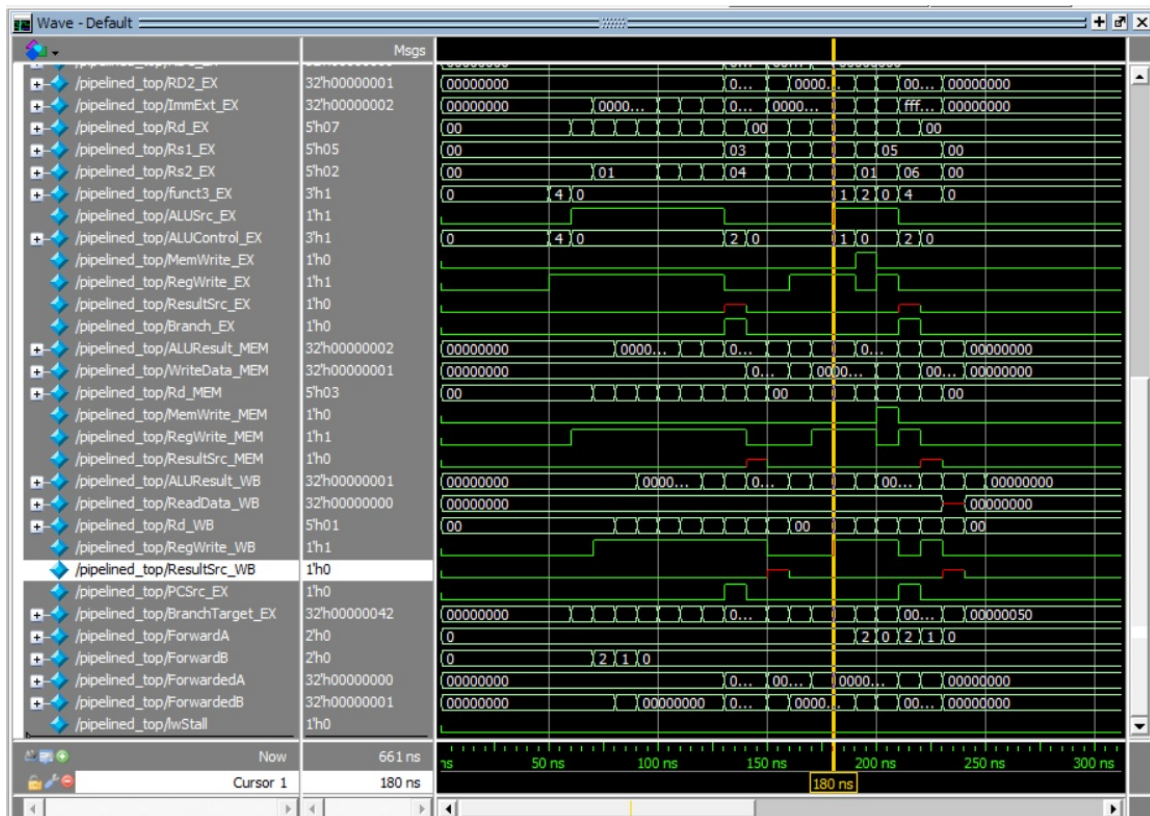


Figure 4.3: Pipelined simulation waveform (EX/MEM/WB signals, forwarding, and hazard signals).

4.4 Cross-Check

The final contents of the data memory produced by the pipelined simulation were compared word-for-word against the single-cycle result and against the expected output supplied with the handout (`mem[0..9] = 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89`¹). Both implementations reproduce the same architectural state, despite the pipelined design taking more *clock cycles* in total (due to the fixed pipeline fill latency and the stall/flush cycles) to reach that state.

¹The handout's reference waveform enumerates ten stored terms; the eleventh value, 89, appears as the final `file/register` snapshot.

Chapter 5

Single-Cycle vs. Pipelined: Comparison

5.1 Qualitative Comparison

Table 5.1: Structural comparison of the two microarchitectures.

Aspect	Single-Cycle	Pipelined (5-stage)
Instructions in flight	1	up to 5
Pipeline registers	none	4 (IF/ID, ID/EX, EX/MEM, MEM/WB)
Extra hazard hardware	none	forwarding unit, hazard detection unit
Branch penalty	none (resolved same cycle)	2 bubble cycles per taken branch
Load-use penalty	none	1 stall cycle
Control unit	1, combinational, full width	1, but its outputs are pipelined forward
Clock period	bounded by the slowest <i>whole</i> instruction	bounded by the slowest <i>single stage</i>

5.2 Quantitative Comparison

Let $T_{c,SC}$ be the single-cycle clock period (sum of the worst-case delay through every stage, dominated by the `lw` path: instruction memory + register file + ALU + data memory + write-back mux), and let $T_{c,PL}$ be the pipelined clock period (bounded by the slowest *individual* stage, in this design the MEM stage or the EX stage depending on synthesis results).

$$T_{c,PL} < T_{c,SC} \quad (\text{ideally } T_{c,PL} \approx T_{c,SC}/5, \text{ in practice higher due to pipeline-register overhead})$$

CPI (cycles per instruction):

- Single-cycle: $\text{CPI}_{SC} = 1$ always.
- Pipelined (ideal, no hazards): $\text{CPI}_{PL} = 1$.
- Pipelined (effective, this design):

$$\text{CPI}_{PL,\text{eff}} = 1 + \frac{(\# \text{ load-use stalls}) \cdot 1 + (\# \text{ taken branches}) \cdot 2}{\# \text{ instructions}}$$

Speedup of the pipelined design over the single-cycle design for a program of N instructions:

$$\text{Speedup} = \frac{N \cdot \text{CPI}_{SC} \cdot T_{c,SC}}{N \cdot \text{CPI}_{PL,\text{eff}} \cdot T_{c,PL}} = \frac{\text{CPI}_{SC} \cdot T_{c,SC}}{\text{CPI}_{PL,\text{eff}} \cdot T_{c,PL}}$$

For the Fibonacci test program, every loop iteration contains two branches (one of which is always taken – either the conditional `beq x3,x4,eq` or the unconditional `beq x4,x4,endlloop`) and no back-to-back load-use dependency (the only load-adjacent instruction pattern in the program is `sw` following `slli`, and stores do not trigger `lwStall`). The pipelined run therefore pays a predictable, branch-dominated stall overhead, while still benefiting from a shorter clock period on every cycle – the mechanism by which pipelining trades instruction *latency* (more cycles per branchy instruction stream) for instruction *throughput* (shorter, more uniform cycles).

5.3 Discussion

The single-cycle design is simpler to design and verify – there is no notion of a hazard because only one instruction is ever “in-flight.” Its cost is a clock period sized for the worst-case instruction, which wastes timing margin on every simpler instruction (e.g. an `add` does not need to wait for a full memory access, but must anyway).

The pipelined design recovers that wasted margin by shortening the clock period to a single stage’s delay, at the cost of the forwarding and hazard-detection hardware described in Chapter 3, and at the cost of a small, bounded per-branch and per-load-use penalty. For instruction streams with few branches and few back-to-back load-dependent instructions, the pipelined design’s throughput advantage dominates; for branch-heavy code (such as the Fibonacci loop, which branches every iteration), the flush penalty reduces – but does not eliminate – the net speedup.

Chapter 6

Design Notes, Assumptions, and Known Limitations

This chapter records assumptions and edge cases identified while reviewing the RTL, so that they are documented rather than left implicit.

1. **Reset polarity.** `PC_reg` and `register_file` use an asynchronous, *active-low* reset (`negedge areset / negedge reset`), and `pipelined_top` wires the top-level `rst` input directly to both. The pipeline registers (`if_id_reg`, `id_ex_reg`, `ex_mem_reg`, `mem_wb_reg`) follow the same convention. Testbenches must drive `rst` low at least once to clear architectural state before releasing it high.
2. **Stall scope.** Only `if_id_reg` and `id_ex_reg` ever stall (`ex_mem_reg` and `mem_wb_reg` have unused `stall` ports tied to `1'b0`). This is intentional: once an instruction is in EX/MEM or MEM/WB, nothing younger can still be waiting behind it, so only the front of the pipeline needs to be held.
3. **Flush scope.** A branch flush (`PCSrc_EX`) clears `if_id_reg` and `id_ex_reg` only; `ex_mem_reg` and `mem_wb_reg` are never flushed, which is correct because the branch itself is already past EX by the time it resolves, and no younger instruction has reached MEM or WB yet.
4. **Interaction of `lwStall` and `PCSrc_EX`.** `id_ex_reg` is flushed on `PCSrc_EX || lwStall` combined; a load-use stall and a branch flush arriving in the same cycle both result in a bubble being injected into ID/EX, which is the correct outcome for either hazard individually. This design does not need to distinguish the two cases in this stage since both require the same corrective action.
5. **Halt behaviour.** `Halt` is defined combinatorially as `Instr_IF == 0` and disables the PC's load-enable (`en = ~Halt & ~lwStall`). Because instruction memory is zero-initialized beyond the loaded program, fetching past the end of the program naturally halts the processor without a dedicated `halt` opcode being decoded by the control unit.
6. **Memory addressing.** Both memories are word-addressed only (`A[31:2]`); there is no byte or halfword load/store support (`lb/lh/sb/sh`), consistent with the RV32I subset targeted by this project (`lw/sw` only).
7. **No branch prediction.** All branches are effectively "predict not-taken," resolved in EX, with a fixed 2-cycle flush penalty when taken – there is no forwarding path that could reduce this penalty further within the given pipeline structure.

Chapter 7

Conclusion

This project implemented and verified two RV32I microarchitectures that share the same functional units but differ in how those units are scheduled in time. The single-cycle design is architecturally simple but clock-rate-limited; the pipelined design recovers clock-rate performance by overlapping instruction execution, at the cost of forwarding and hazard-detection hardware and a small, bounded stall/flush penalty on loads and branches. Both designs were verified against the same Fibonacci test program and produce identical final architectural state, confirming that the pipelining transformation preserves program correctness while improving throughput.

Appendix A

Module Interfaces

Table A.1: Summary of RTL modules and their role in the pipeline.

Module	Stage	Role
PC_reg	IF	Program counter register + next-PC mux
instr_mem	IF	Instruction ROM (64×32 -bit)
if_id_reg	IF/ID	Pipeline register
register_file	ID	32×32 -bit register bank
sign_extend	ID	Immediate generation
control_unit	ID	Main decoder + ALU decoder
id_ex_reg	ID/EX	Pipeline register
hazard_detection_unit	ID/EX	Load-use stall detection
forward_unit	EX	EX/MEM and MEM/WB forwarding
ALU	EX	Arithmetic/logic execution
branch_unit	EX	Branch condition + target computation
ex_mem_reg	EX/MEM	Pipeline register
data_mem	MEM	Data RAM (64×32 -bit)
mem_wb_reg	MEM/WB	Pipeline register
mux	(shared)	Generic 32-bit 2:1 multiplexer

Appendix B

Selected RTL Listings

B.1 Hazard Detection Unit

```
1 module hazard_detection_unit(  
2     input  wire      ResultSrc_EX ,  
3     input  wire [4:0] Rd_EX ,  
4     input  wire [4:0] Rs1_ID ,  
5     input  wire [4:0] Rs2_ID ,  
6     output wire      lwStall  
7 );  
8     assign lwStall = ResultSrc_EX &&  
9                 (Rd_EX != 5'b0) &&  
10                ((Rd_EX == Rs1_ID) || (Rd_EX == Rs2_ID));  
11 endmodule
```

B.2 Forwarding Unit

```
1 module forward_unit(  
2     input  wire [4:0] Rs1_EX ,  
3     input  wire [4:0] Rs2_EX ,  
4     input  wire [4:0] Rd_MEM ,  
5     input  wire      RegWrite_MEM ,  
6     input  wire [4:0] Rd_WB ,  
7     input  wire      RegWrite_WB ,  
8     output reg [1:0] ForwardA ,  
9     output reg [1:0] ForwardB  
10 );  
11     always @(*) begin  
12         if (RegWrite_MEM && (Rd_MEM != 5'b0) && (Rd_MEM == Rs1_EX))  
13             ForwardA = 2'b10;  
14         else if (RegWrite_WB && (Rd_WB != 5'b0) && (Rd_WB == Rs1_EX))  
15             ForwardA = 2'b01;  
16         else  
17             ForwardA = 2'b00;  
18  
19         if (RegWrite_MEM && (Rd_MEM != 5'b0) && (Rd_MEM == Rs2_EX))  
20             ForwardB = 2'b10;  
21         else if (RegWrite_WB && (Rd_WB != 5'b0) && (Rd_WB == Rs2_EX))  
22             ForwardB = 2'b01;  
23         else  
24             ForwardB = 2'b00;  
25     end
```

```
26 endmodule
```

B.3 Branch Unit

```
1 module branch_unit(  
2     input  wire      Branch_EX ,  
3     input  wire [2:0] funct3_EX ,  
4     input  wire      Zero ,  
5     input  wire      sign ,  
6     input  wire [31:0] PC_EX ,  
7     input  wire [31:0] ImmExt_EX ,  
8     output reg       PCSrc_EX ,  
9     output wire [31:0] BranchTarget_EX  
10 );  
11     assign BranchTarget_EX = PC_EX + ImmExt_EX;  
12  
13     always @(*) begin  
14         case (funct3_EX)  
15             3'b000: PCSrc_EX = Branch_EX & Zero;    // beq  
16             3'b001: PCSrc_EX = Branch_EX & ~Zero;  // bne  
17             3'b100: PCSrc_EX = Branch_EX & sign;    // blt  
18             default: PCSrc_EX = 1'b0;  
19         endcase  
20     end  
21 endmodule
```